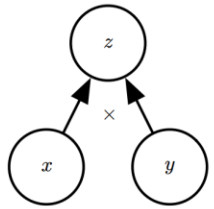
Lecture 2B: Backpropagation using Computational Graphs

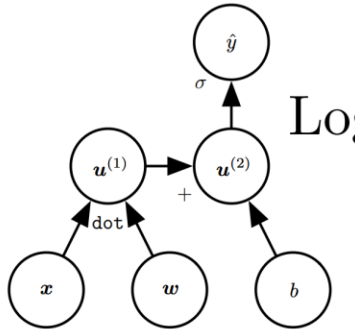
KEVIN BRYSON

What are computational graphs ?

Multiplication



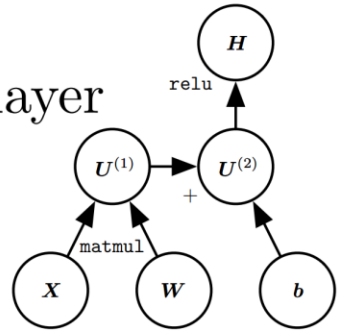
(a)



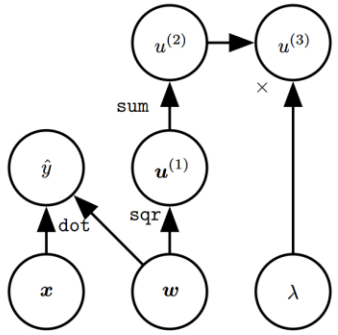
Logistic regression

(b)

ReLU layer



(c)

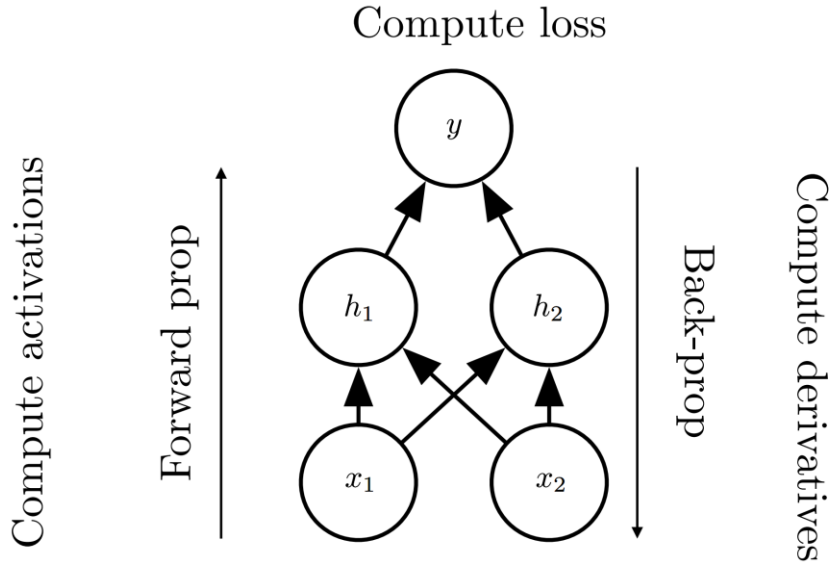


Linear regression and weight decay

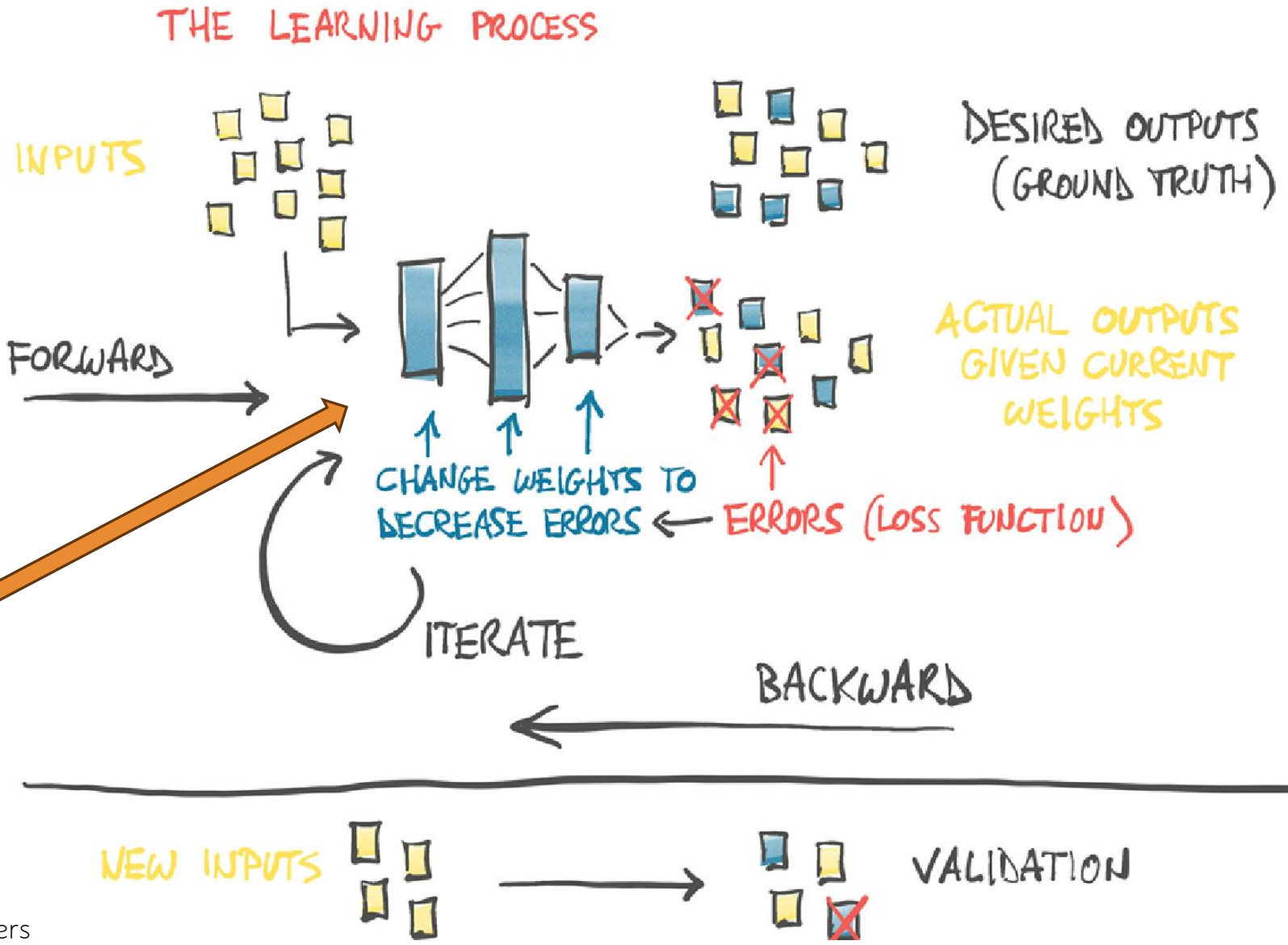
(d)

Backpropagation on a computational graph

1. For each sample, make a prediction (*forward pass*)
   - States of units (i.e. values) are propagated from the input layer to the output layer when doing “out = model(x)”.
2. Measure the output error of the model (i.e. the loss).
3. Go back through each layer to measure error contribution (i.e. gradient of the loss) from each connection (*reverse pass*)
   - Gradients of the error are propagated backwards from the output layer to the input layer
4. Make a small change to weights in the negative gradient direction to reduce the output error (*gradient descent*)



In PyTorch



The Computational Graph

Image from *Deep Learning with PyTorch*, Manning Publishers

Chain rule of calculus

We can use the chain rule of calculus to compute the derivatives of functions formed by composing other functions with known derivatives.

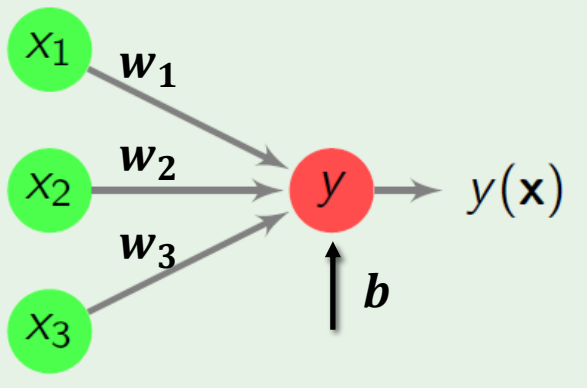
Suppose $y = g(x), z = f(g(x)) = f(y)$, then the chain rule states that

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

This can be generalised beyond the scalar case to vectors y and x:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j}\frac{\partial y_j}{\partial x_i}$$

Gradient descent for our neuron



$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

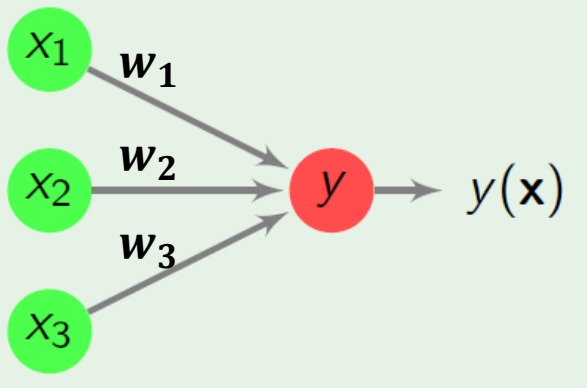
Gradient of the loss:

$$\nabla_{Wb} L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \frac{\partial L}{\partial b}\right]$$

Update:

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i}$$

Chain rule for our neuron



$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

Derivative of loss function

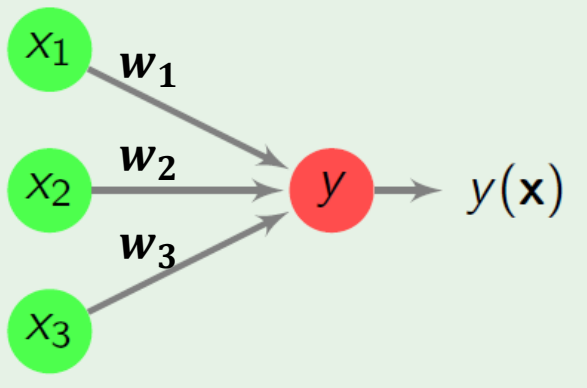
Derivative of activation function

Derivative of loss wrt to weight

Derivative of Linear component

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Loss derivative



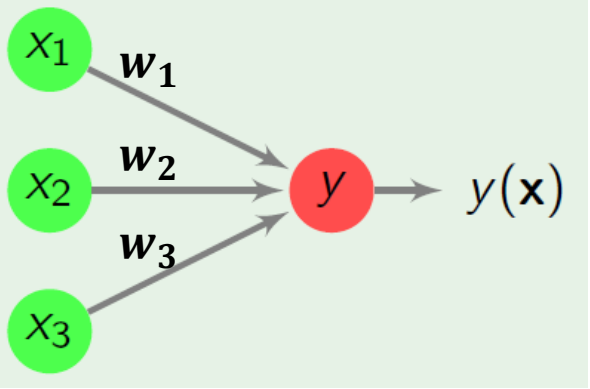
$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$\frac{\partial L}{\partial y} = \frac{\partial}{\partial y}\frac{1}{2}(y - t)^2 = y - t$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

Activation derivative



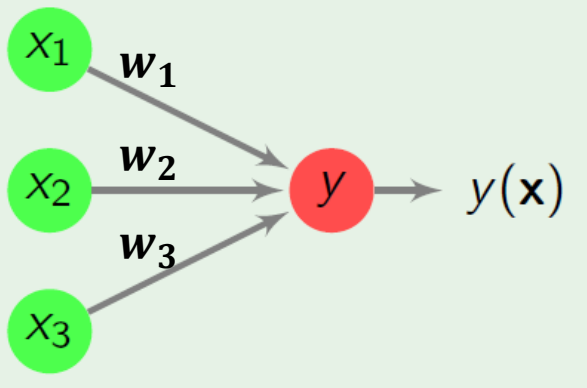
$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$\frac{\partial y}{\partial z} = \phi'(z)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

Linear part derivative



$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

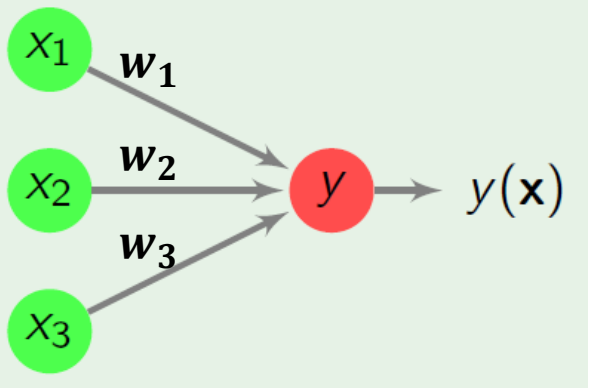
$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

$$\frac{\partial z}{\partial w_i} = \frac{\partial}{\partial w_i} b + \sum_i w_i x_i = x_i$$

$$\frac{\partial z}{\partial b} = \frac{\partial}{\partial b} b + \sum_i w_i x_i = 1$$

Putting it all together



$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

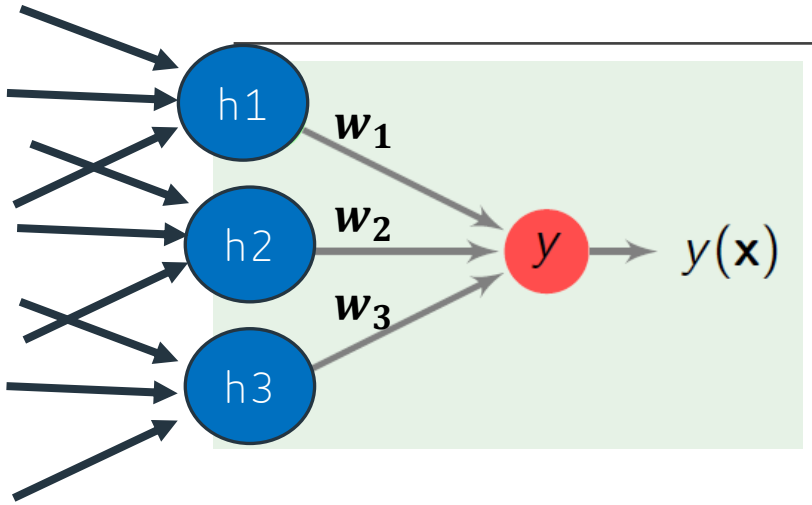
$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i x_i}^{z}\right)$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

$$\frac{\partial L}{\partial w_i} = x_i \cdot \phi'(z) \cdot [y - t]$$

$$\frac{\partial L}{\partial b} = \phi'(z) \cdot [y - t]$$

Now imagine that the previous layer was a hidden layer with neuron values h1, h2, h3 ...



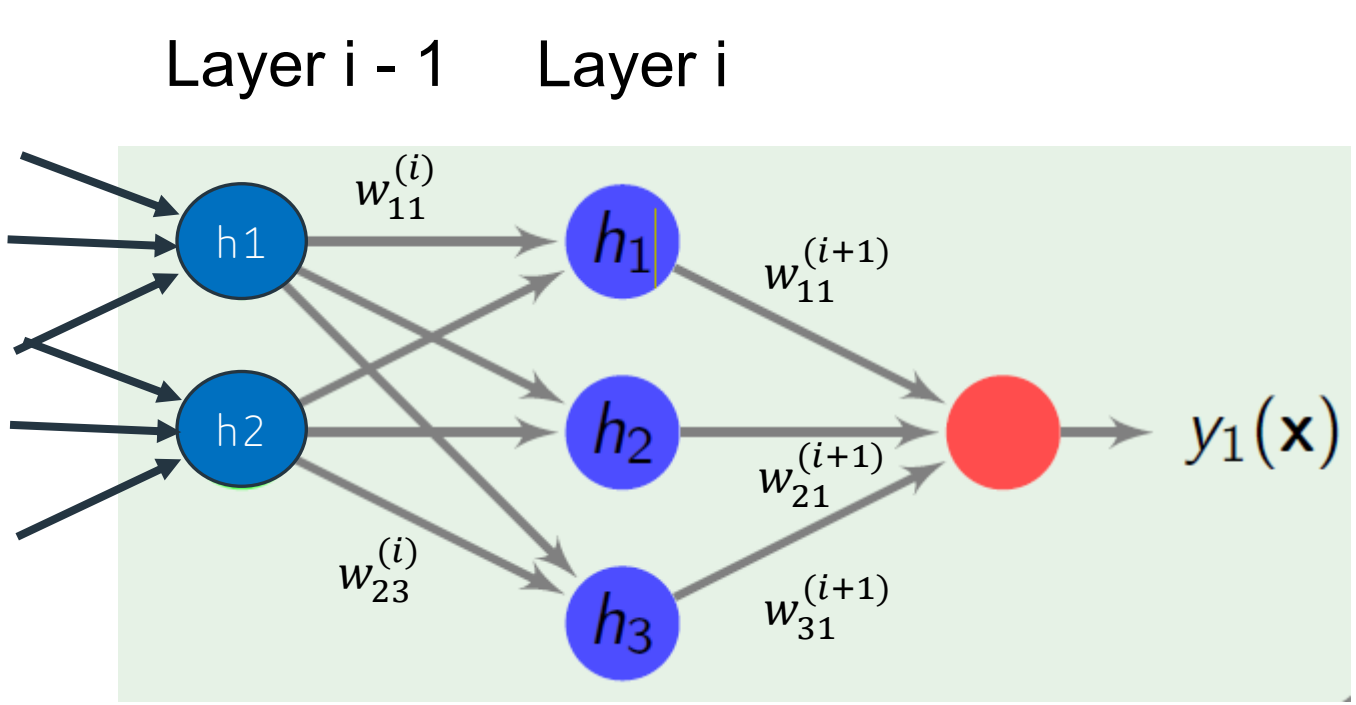
$$\frac{\partial L}{\partial h_i} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial h_i}$$

$$y(\mathbf{x}) = \phi\left(\overbrace{b + \sum_i w_i h_i}^{z}\right)$$

$$\frac{\partial L}{\partial h_i} = w_i \, \phi'(z) \cdot [y - t]$$

$$L(\mathbf{x}, t) = \frac{1}{2}(y(\mathbf{x}) - t)^2$$

What happens with a deep network?



Derivative of loss function

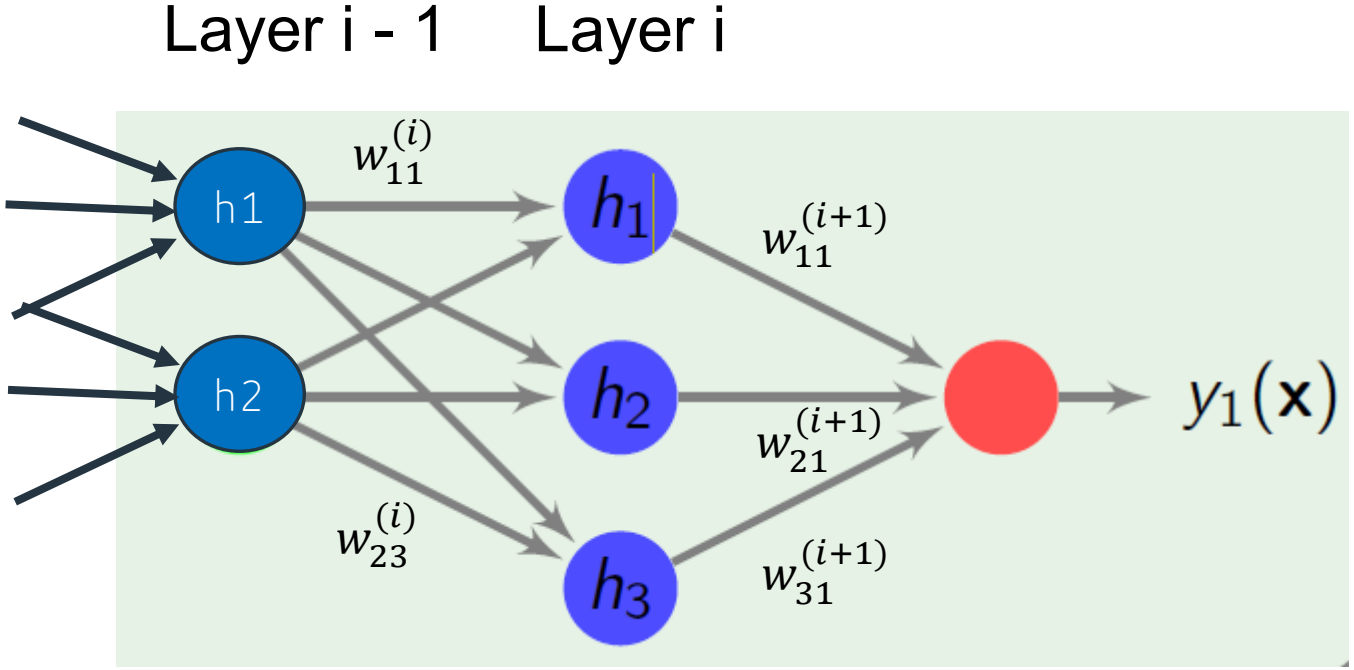
Derivative of activation function

Derivative of loss wrt to weight

Derivative of Linear component

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

What happens with a deep network?



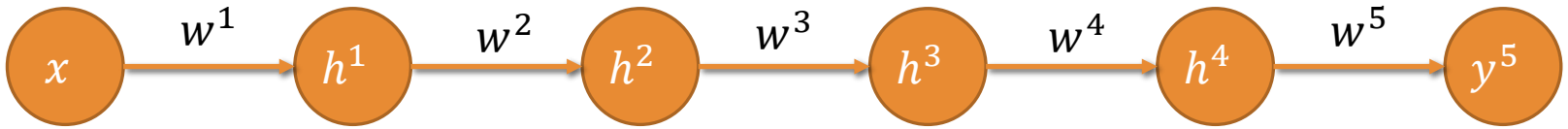
The derivative of the loss at a hidden unit is the **sum of the derivatives** from the neurons it contributes to **in the next layer**.

$$\frac{\partial L}{\partial h_j^{(i-1)}} = \sum_k w_{jk}^{(i)} \emptyset'(z_k^{(i)}) \frac{\partial L}{\partial h_k^{(i)}}$$

Why knowing this theory is useful … the vanishing gradient problem

Consider a simple deep neural network:

- 5 layers
- A single neuron per layer
- How does an error propagate?



The vanishing gradient problem

$$\frac{\partial L}{\partial w^5} = [y^5 - t] \cdot \phi'(z^5) \cdot h^4$$

$$\frac{\partial L}{\partial w^4} = \left[[y^5 - t] \cdot \phi'(z^5) \cdot w^5\right] \cdot \phi'(z^4) \cdot h^3$$

$$\frac{\partial L}{\partial w^3} = \left[[y^5 - t] \cdot \phi'(z^5) \cdot w^5 \cdot \phi'(z^4) \cdot w^4\right] \cdot \phi'(z^3) \cdot h^2$$

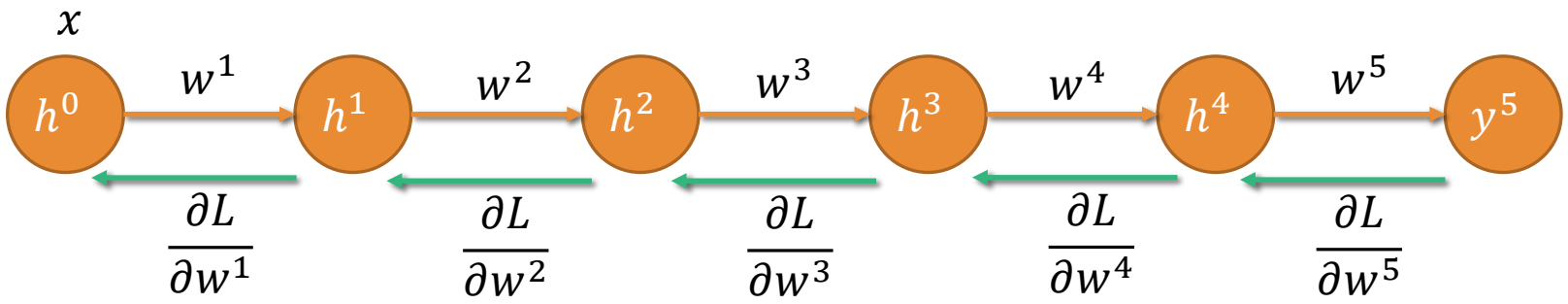
$$\frac{\partial L}{\partial w^2} = \left[[y^5 - t] \cdot \phi'(z^5) \cdot w^5 \cdot \phi'(z^4) \cdot w^4 \cdot \phi'(z^3) \cdot w^3\right] \cdot \phi'(z^2) \cdot h^1$$

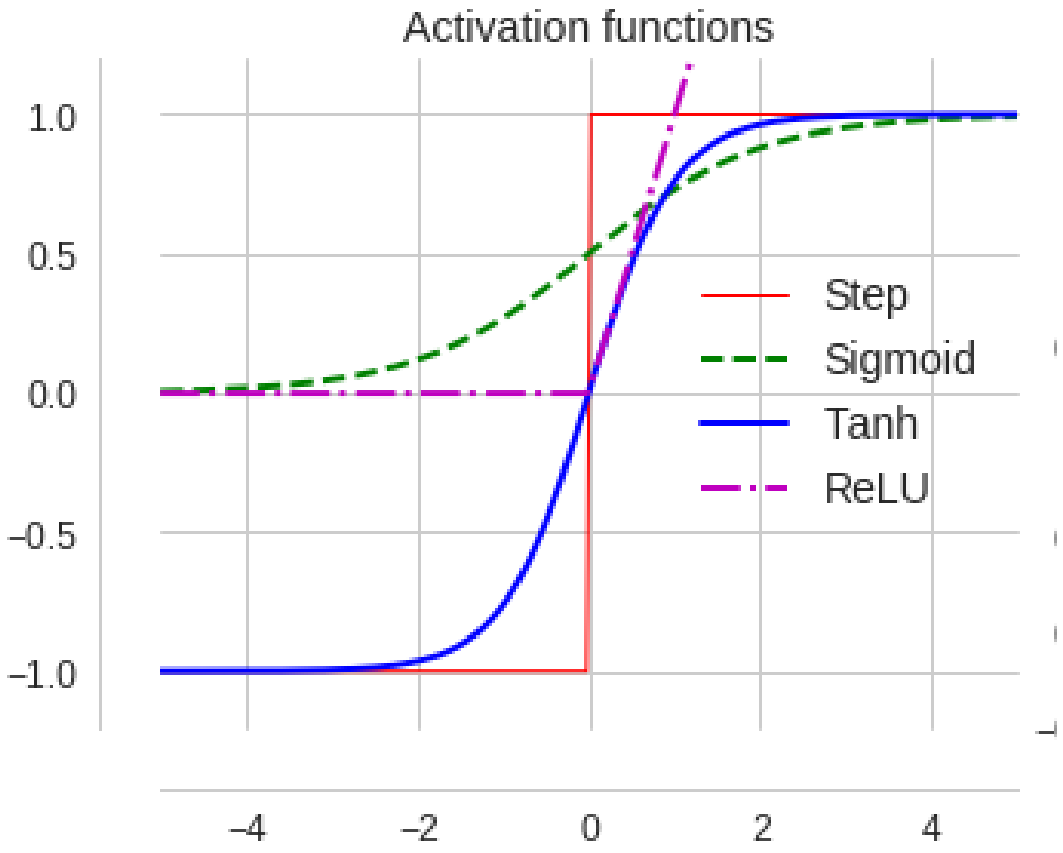
$$\frac{\partial L}{\partial w^1} = [y^5 - t] \cdot \phi'(z^5) \cdot w^5 \cdot \phi'(z^4) \cdot w^4 \cdot \phi'(z^3) \cdot w^3 \cdot \phi'(z^2) \cdot w^2 \cdot \phi'(z^1) \cdot x$$

The vanishing gradient problem

Similarly, for n layers we have:

$$\frac{\partial L}{\partial w^i} = [y^n - t] \cdot \phi'(z^i) \cdot h^{i-1} \cdot \prod_{j=i+1}^{n} \phi'(z^j) \cdot w^j$$





The vanishing gradient problem

- Multiplying by numbers <1 reduces the gradient
- The equation multiplies by the activation derivative **at each layer**
- If the activation is saturated <1 !
- Hence using ReLU (which does not saturate for z>0) reduces the problem
- Still an issue if weights are small (which we generally want!) → residual networks

How do we calculate these derivatives (or gradients) in practice on a computer?

There are many ways to calculate derivatives or gradients in computer science:

- Numerical calculation …
- Symbolic calculation …
- Forward propagation …
- **Back propagation …**

Numerical Differentiation

The simplest approach is to compute an approximation of derivative numerically.

$$\begin{aligned} h'(x_0) &= \lim_{x \to x_0} \frac{h(x) - h(x_0)}{x - x_0} \\ &= \lim_{\epsilon \to 0} \frac{h(x_0 + \epsilon) - h(x_0)}{\epsilon} \end{aligned}$$

But this tends to lead to imprecise results (getting worse for more complex functions). It also scales poorly, as you would call $f(x)$ $N + 1$ times for N parameters, making it too inefficient for deep networks.

It is, however, useful for double-checking other methods.

Symbolic Differentiation

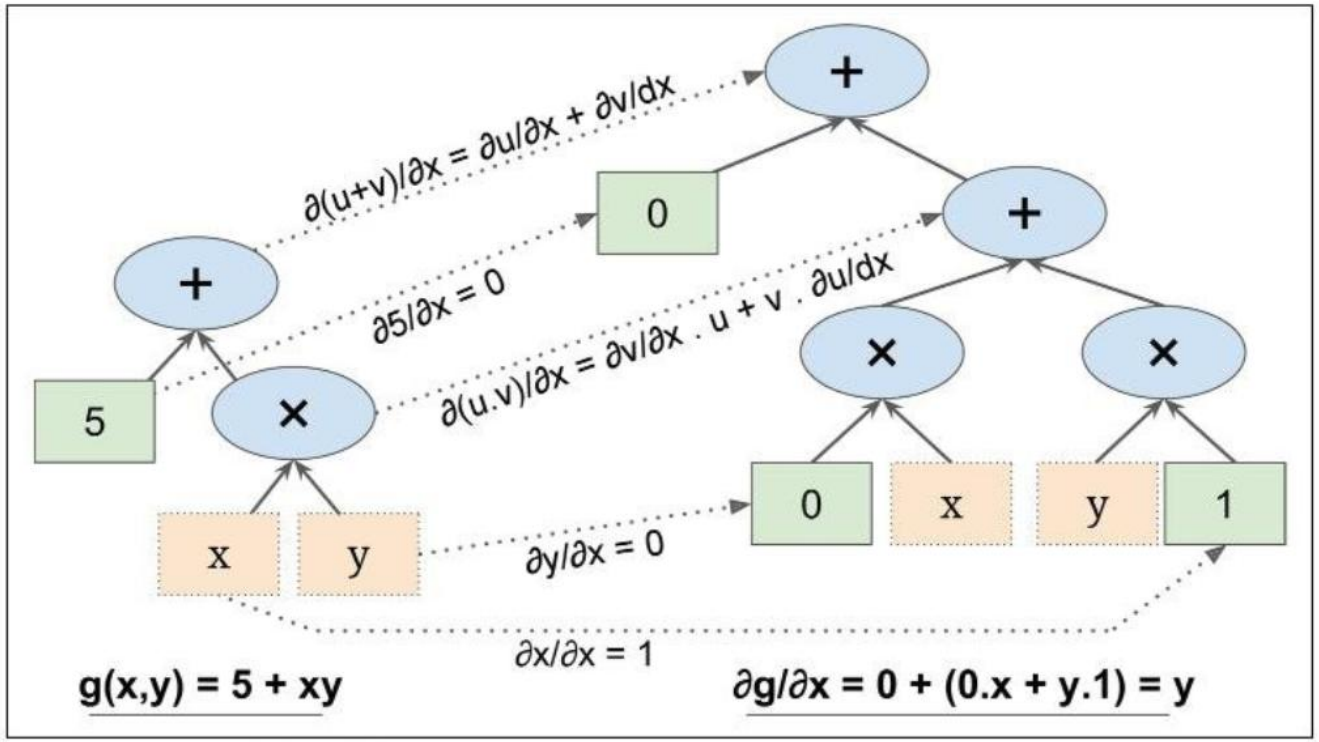


Figure D-1. Symbolic differentiation

ISSUES:

1. Expression length can grow exponentially.

2. With millions of parameters … Would need to store and calculate millions of expressions.

Figure D-1 shows how symbolic differentiation works on an even simpler function, $g(x,y) = 5 + xy$. The graph for that function is represented on the left. After symbolic differentiation, we get the graph on the right, which represents the partial derivative $\frac{\partial g}{\partial x} = 0 + (0 \times x + y \times 1) = y$ (we could similarly obtain the partial derivative with regards to y).

Forward-mode autodiff

Differentiating $x^2 y + y + 2$ for $x = 3$ and $y = 4$.

ISSUE:

With millions of parameters … would need to do millions of forward calculations of the network.

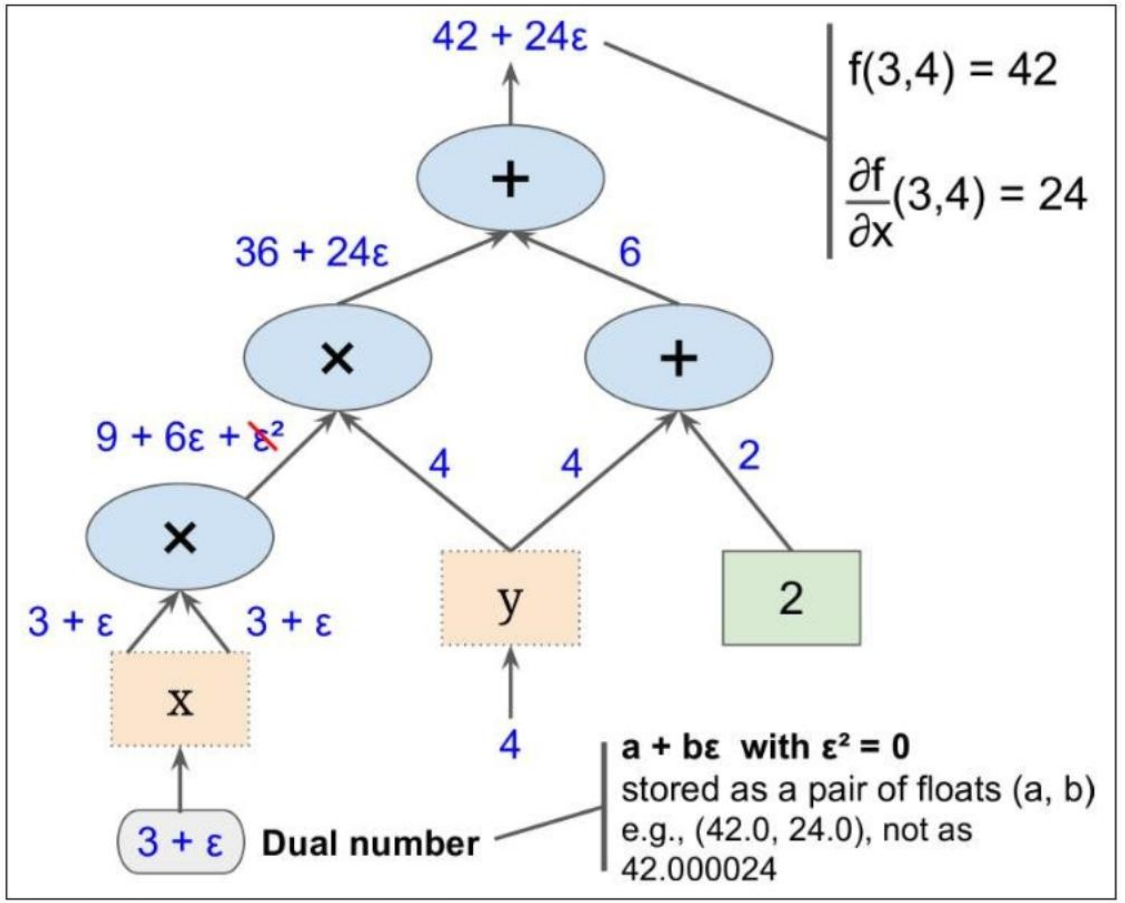


Figure D-2. Forward-mode autodiff

Reverse-mode autodiff

Differentiating $x^2 y + y + 2$ for $x = 3$ and $y = 4$.

WORKS FOR MILLIONS OF PARAMETERS !

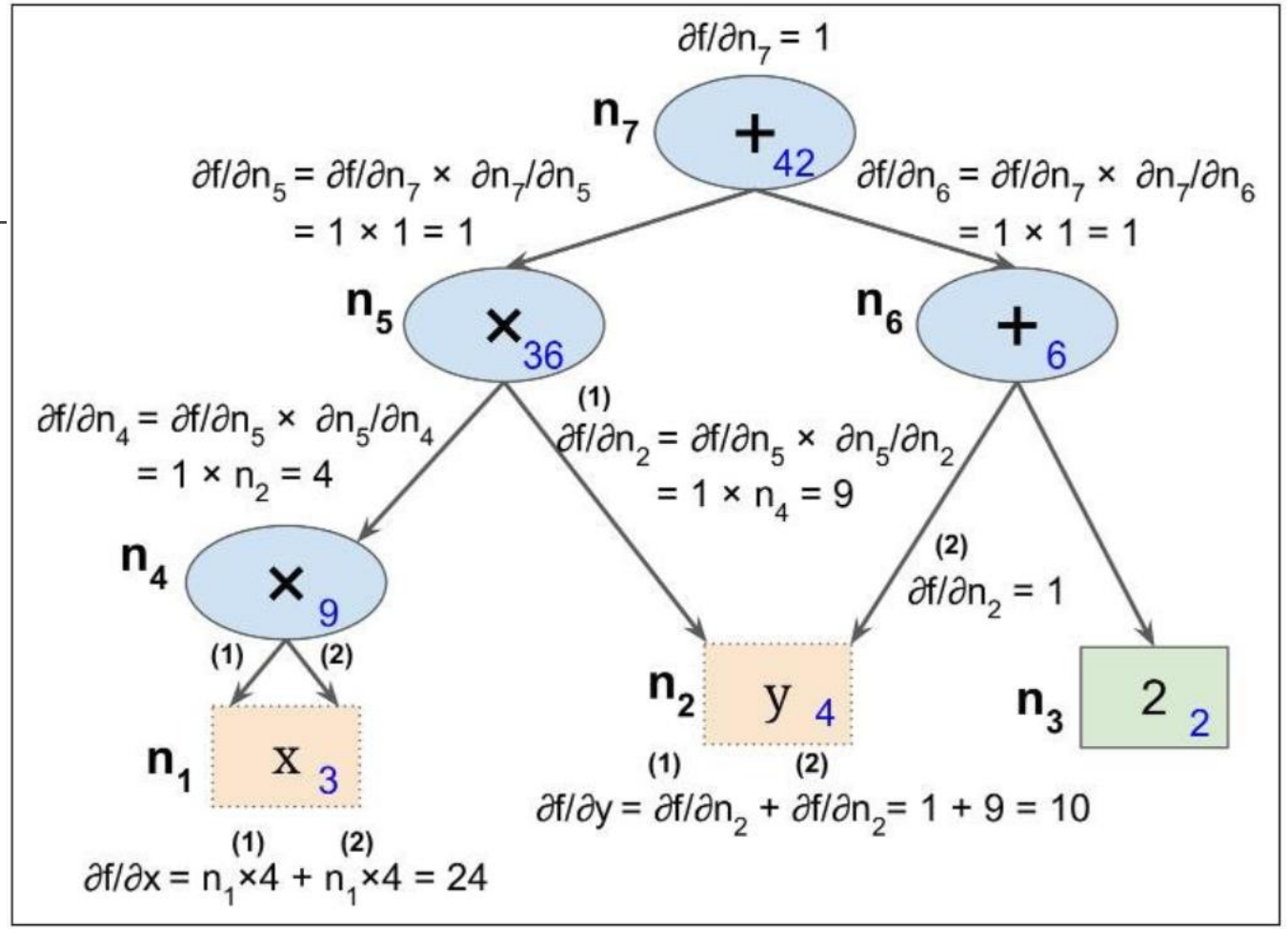


Figure D-3. Reverse-mode autodiff

PyTorch builds a dynamic graph

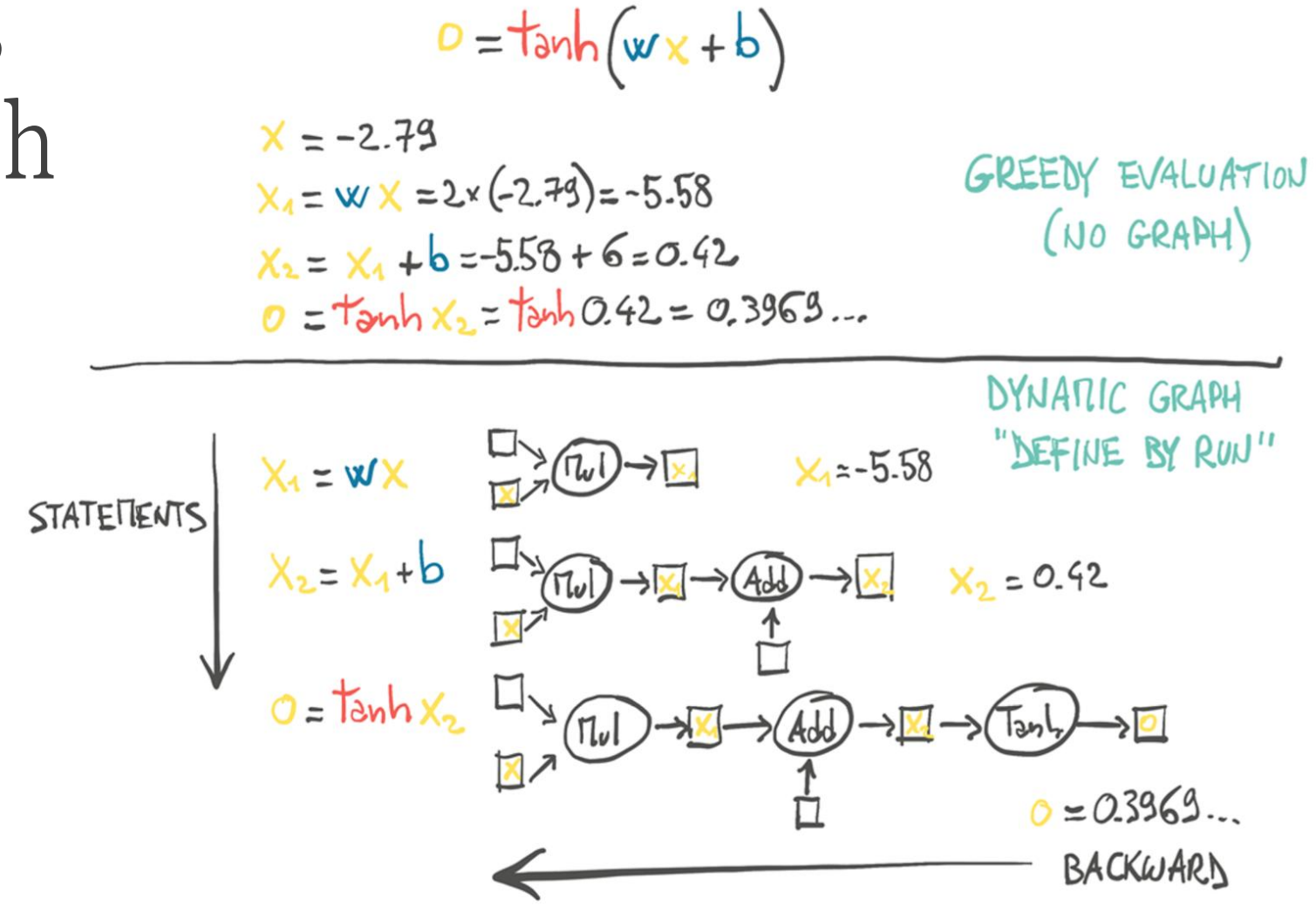


Figure 1.3 Dynamic graph for a simple computation corresponding a single neuron.

Autograd in pytorch:

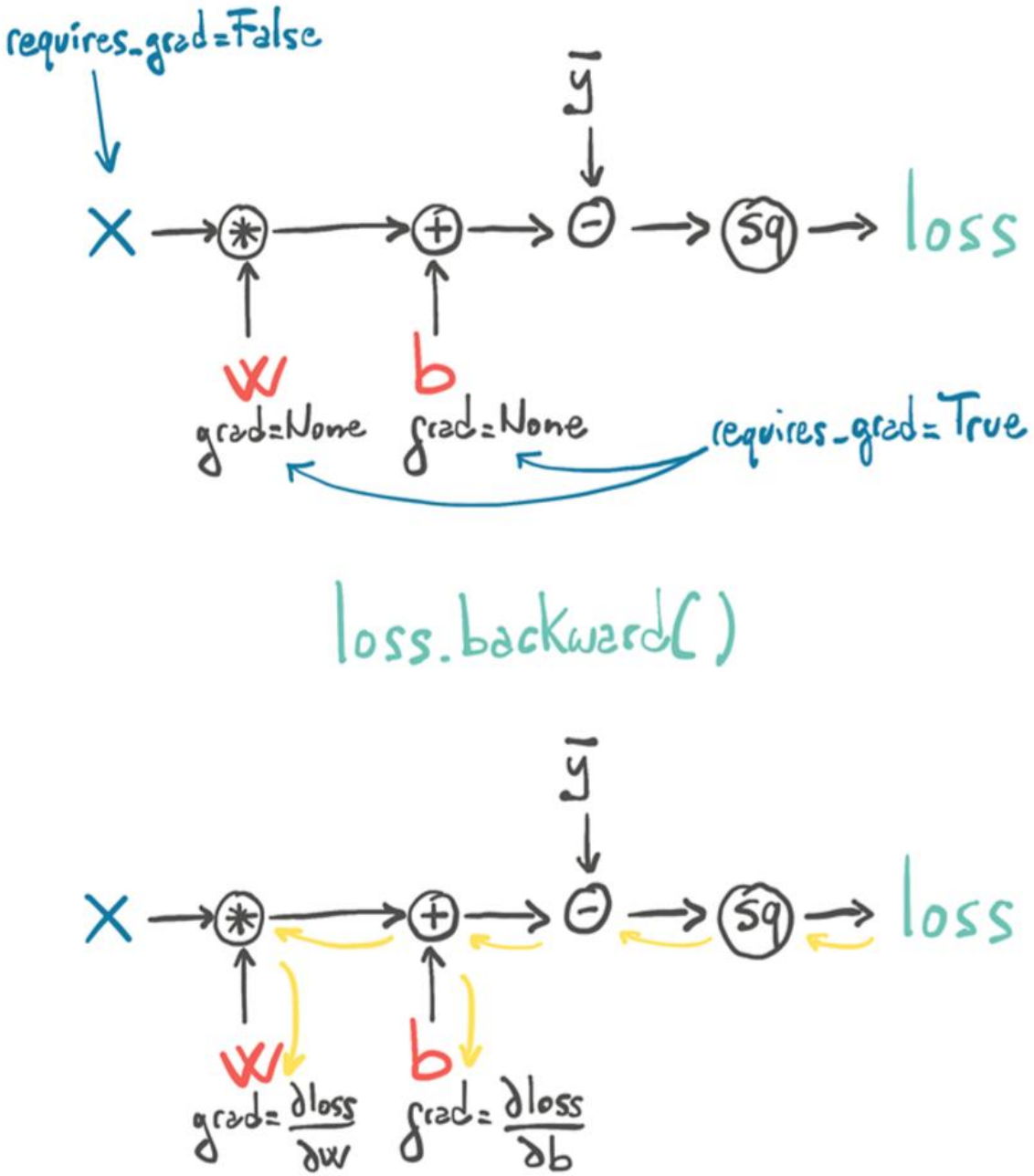
```
def model(x, w, b):
    return w * x + b

def loss_fn(y, y_bar):
    squared_diffs = (y - y_bar)**2
    return squared_diffs.mean()

params = torch.tensor([1.0, 0.0], requires_grad=True)

loss = loss_fn(model(x, *params), y_bar)
loss.backward()
params.grad
```

tensor([4517.2969, 82.6000])



Vector formulation

Jacobian-gradient operations

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$$

which, in vector notation is,

$$\nabla_x z = \left(\frac{\partial y}{\partial x}\right)^T \nabla_y z,$$

where $\frac{\partial y}{\partial x}$ is the $n \times m$ *Jacobian* matrix of g.
The gradient of a variable x is obtained by multiplying a Jacobian $\frac{\partial y}{\partial x}$ by a gradient $\nabla_y z$,.
Backpropagation performs this Jacobian-gradient product for each operation in the graph.

Forward pass

Algorithm 6.3 Forward propagation through a typical deep neural network and the computation of the cost function. The loss $L(\hat{\boldsymbol{y}}, \boldsymbol{y})$ depends on the output $\hat{\boldsymbol{y}}$ and on the target $\boldsymbol{y}$ (see section 6.2.1.1 for examples of loss functions). To obtain the total cost J, the loss may be added to a regularizer $\Omega(\theta)$, where θ contains all the parameters (weights and biases). Algorithm 6.4 shows how to compute gradients of J with respect to parameters $\boldsymbol{W}$ and $\boldsymbol{b}$. For simplicity, this demonstration uses only a single input example $\boldsymbol{x}$. Practical applications should use a minibatch. See section 6.5.7 for a more realistic demonstration.

Require: Network depth, l
Require: $\boldsymbol{W}^{(i)}, i \in \{1, \ldots, l\}$, the weight matrices of the model
Require: $\boldsymbol{b}^{(i)}, i \in \{1, \ldots, l\}$, the bias parameters of the model
Require: $\boldsymbol{x}$, the input to process
Require: $\boldsymbol{y}$, the target output

$\boldsymbol{h}^{(0)} = \boldsymbol{x}$
for $k = 1, \ldots, l$ **do**
  $\boldsymbol{a}^{(k)} = \boldsymbol{b}^{(k)} + \boldsymbol{W}^{(k)} \boldsymbol{h}^{(k-1)}$
  $\boldsymbol{h}^{(k)} = f(\boldsymbol{a}^{(k)})$
end for
$\hat{\boldsymbol{y}} = \boldsymbol{h}^{(l)}$
$J = L(\hat{\boldsymbol{y}}, \boldsymbol{y}) + \lambda \Omega(\theta)$

Backward pass

Algorithm 6.4 Backward computation for the deep neural ne
rithm 6.3, which uses, in addition to the input $\boldsymbol{x}$, a target $\boldsymbol{y}$. Thi
yields the gradients on the activations $\boldsymbol{a}^{(k)}$ for each layer k, sta
output layer and going backwards to the first hidden layer. From t
which can be interpreted as an indication of how each layer's output should change to reduce error, one can obtain the gradient on the parameters of each layer. The gradients on weights and biases can be immediately used as part of a stochastic gradient update (performing the update right after the gradients have been computed) or used with other gradient-based optimizatio

After the forward computation, compute the gradient o
$\boldsymbol{g} \leftarrow \nabla_{\hat{\boldsymbol{y}}} J = \nabla_{\hat{\boldsymbol{y}}} L(\hat{\boldsymbol{y}}, \boldsymbol{y})$
for $k = l, l-1, \ldots, 1$ **do**
Convert the gradient on the layer's output into a gradient on the pre-nonlinearity activation (element-wise multiplication if f is element-wise):
$\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{a}^{(k)}} J = \boldsymbol{g} \odot f'(\boldsymbol{a}^{(k)})$
Compute gradients on weights and biases (including the regularization term, where needed):
$\nabla_{\boldsymbol{b}^{(k)}} J = \boldsymbol{g} + \lambda \nabla_{\boldsymbol{b}^{(k)}} \Omega(\theta)$
$\nabla_{\boldsymbol{W}^{(k)}} J = \boldsymbol{g}\, \boldsymbol{h}^{(k-1)\top} + \lambda \nabla_{\boldsymbol{W}^{(k)}} \Omega(\theta)$
Propagate the gradients w.r.t. the next lower-level hidden layer's activations:
$\boldsymbol{g} \leftarrow \nabla_{\boldsymbol{h}^{(k-1)}} J = \boldsymbol{W}^{(k)\top} \boldsymbol{g}$
end for

$$\frac{\partial L}{\partial h_i} = w_i \; \phi'(z) \cdot [y - t]$$

$$\frac{\partial L}{\partial h_j^{(i)}} = \sum_k w_{jk}^{(i+1)} \emptyset'(z_k^{(i+1)}) \frac{\partial L}{\partial h_k^{(i+1)}}$$

Goodfellow Book explains the back-propagation process in detail.

Reading

Again, if you are uncertain about the maths involved in this lecture then it might be good to refresh this with Appendix A, B and C of Simon's book.

Otherwise much more detail about calculating gradients for deep neural networks can be found in Chapter 7 of Simon's book.

A more mathematically and algorithmically formal description (as indicated by the last two slides!) is given by:

Chapter 6 of Goodfellow et al. 2018. https://www.deeplearningbook.org/contents/mlp.html